

```

9   Calculator cal;
10
11  @Before
12  /*
13   the init() method will be called for each test, such
14   testAdd() as well as testSub()
15   */
16  public void init() {
17
18      cal = new Calculator();
19
20  }
21
22  @Test
23  public void testAdd() {
24
25      int x = 10;
26      int y = 20;
27      assertEquals(30, cal.add(x, y));
28
29  }
30
31  @Test
32  public void testSub() {
33
34      int x = 10;
35      int y = 20;
36      assertEquals(-10, cal.sub(x, y));
37
38  }

```

Parameterised unit test cases using JUnit

If you want to test any method with multiple input values, you would normally have to write multiple test cases for the same method. But if you use the parameterised unit testing technique, you don't need to write multiple test cases for the same method.

Let's look at the example of the *Calculator* class defined in Code-5. If you have to create parameterised test cases for the *add()* method of the *Calculator* class with multiple inputs, then consider the following code for that requirement.

Code-7

```

1   package com.emertxe;
2
3   import static org.junit.Assert.*;
4   import java.util.Arrays;
5   import java.util.Collection;
6   import org.junit.Assert;
7   import org.junit.Before;
8   import org.junit.Test;
9   import org.junit.runner.RunWith;
10  import org.junit.runners.Parameterized;
11  import org.junit.runners.Parameterized.Parameters;

```

```

12
13  @RunWith(Parameterized.class)
14  public class AddParamTest {
15
16      private int expectedResult;
17      private int firstVal;
18      private int secondVal;
19      Calculator cal;
20
21      @Before
22      public void init() {
23
24          cal = new Calculator();
25      }
26
27      public AddParamTest(int expectedResult, int firstVal, int
28      secondVal) {
29
30          this.expectedResult = expectedResult;
31          this.firstVal = firstVal;
32          this.secondVal = secondVal;
33      }
34
35      @Parameters
36      public static Collection<Object[]> testData() {
37
38          Object[][] data = new Object[][] { { 6, 2, 4 }, { 7, 4, 3 },
39          { 8, 2, 6 } };
40
41          return Arrays.asList(data);
42      }
43
44      @Test
45      public void testAdd() {
46          Assert.assertEquals(expectedResult, cal.add(firstVal,
47          secondVal));
48      }
49  }

```

When the test case written in Code-7 is executed, then an execution occurs in the following order:

1. Parameterised class at Line 11 is executed.
2. Static method at Line 32 is executed.
3. Instance of *AddParamTest* class at Line 12 is executed.
4. The data {6,2,4}, {7,4,3} and {8,2,6} at Lines 34-35 is passed to the constructor at Line 24.
5. *testAdd()* method at Line 41 is executed. 

By: Vikas Kumar Gautam

The author is a mentor at Emerx Information Technology (P) Ltd. His main areas of expertise include application development using Java/J2EE and Android for both Web and mobile devices. A Sun Certified Java Professional (SCJP), his interests include acquiring greater expertise in the application space by learning from the latest happenings in the industry. He can be reached at vikash_kumar@emertxe.com